

Experiment 'p_4_2_c_cumulative_totals_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_4_2_c_cumulative_totals_seq

Natural language statement: null

Method signature: p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)

Ensures

- $|totals| == |inputs|$
- forall $i :: 0 \leq i < |inputs| \implies totals[i] == (if\ i == 0\ then\ inputs[0]\ else\ totals[i - 1] + inputs[i])$

Requires

- $0 < |inputs|$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)`

The method should respect the following contract:

requires $0 < |\text{inputs}|$, ensures $|\text{totals}| == |\text{inputs}|$, ensures forall $i :: 0 \leq i < |\text{inputs}|$
 $\hookrightarrow | ==> \text{totals}[i] == (\text{if } i == 0 \text{ then } \text{inputs}[0] \text{ else } \text{totals}[i - 1] + \text{inputs}[i])$

Produce and show only the Dafny body of this method, including the curly braces that
 $\hookrightarrow$ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 $\hookrightarrow$ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLKpSn6At7YODDa4zUcJ5Ytli0VN

Model: gpt-5.5-2026-04-23

Created at: 1780652455

Finish reason: stop

Usage: 358 tokens in, and 3124 tokens out

```
{
  totals := [inputs[0]];
  var n := |inputs|;
  var k := 1;

  while k < n
    invariant n == |inputs|
    invariant 1 <= k <= n
    invariant |totals| == k
    invariant forall j :: 0 <= j < k ==> totals[j] == (if j == 0 then inputs[0] else
       $\hookrightarrow$  totals[j - 1] + inputs[j])
    decreases n - k
  {
    var prevTotals := totals;
    assert |prevTotals| == k;
    assert forall j :: 0 <= j < k ==> prevTotals[j] == (if j == 0 then inputs[0] else
       $\hookrightarrow$  prevTotals[j - 1] + inputs[j]);

    var next := prevTotals[k - 1] + inputs[k];
    totals := prevTotals + [next];
  }
```

```

forall j | 0 <= j < k
  ensures totals[j] == prevTotals[j]
{
  assert 0 <= j < |prevTotals|;
  assert totals == prevTotals + [next];
  assert totals[j] == (prevTotals + [next])[j];
  assert (prevTotals + [next])[j] == prevTotals[j];
}

forall j | 0 <= j < k + 1
  ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
{
  if j < k {
    assert totals[j] == prevTotals[j];
    if j == 0 {
      assert prevTotals[j] == inputs[0];
    } else {
      assert 0 <= j - 1 < k;
      assert totals[j - 1] == prevTotals[j - 1];
      assert prevTotals[j] == prevTotals[j - 1] + inputs[j];
    }
  } else {
    assert j == k;
    assert 0 < j;
    assert j - 1 == k - 1;
    assert totals[j] == next;
    assert totals[j - 1] == prevTotals[k - 1];
    assert next == prevTotals[k - 1] + inputs[k];
  }
}

k := k + 1;
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_4_2_c_cumulative_totals_seq/program.dfy(39,10): Error: assertion might not
  ↪ hold
  |
39 | assert prevTotals[j] == inputs[0];
  | ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature

↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLM0tIHLFS8avdfY8Vn8bH6L6LHg

Model: gpt-5.5-2026-04-23

Created at: 1780652528

Finish reason: stop

Usage: 1016 tokens in, and 2873 tokens out

```
{
  totals := [inputs[0]];
  var n := |inputs|;
  var k := 1;

  while k < n
    invariant n == |inputs|
    invariant 1 <= k <= n
    invariant |totals| == k
    invariant forall j :: 0 <= j < k ==> totals[j] == (if j == 0 then inputs[0] else
      ↪ totals[j - 1] + inputs[j])
    decreases n - k
  {
    var prevTotals := totals;
    assert |prevTotals| == k;
    assert forall j :: 0 <= j < k ==> prevTotals[j] == (if j == 0 then inputs[0] else
      ↪ prevTotals[j - 1] + inputs[j]);

    var next := prevTotals[k - 1] + inputs[k];
    totals := prevTotals + [next];

    forall j | 0 <= j < k
      ensures totals[j] == prevTotals[j]
    {
      assert 0 <= j < |prevTotals|;
      assert totals == prevTotals + [next];
      assert totals[j] == (prevTotals + [next])[j];
      assert (prevTotals + [next])[j] == prevTotals[j];
    }

    forall j | 0 <= j < k + 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      if j < k {
        assert 0 <= j < k;
        assert totals[j] == prevTotals[j];
        assert prevTotals[j] == (if j == 0 then inputs[0] else prevTotals[j - 1] + inputs[
          ↪ j]);
        if j == 0 {
          assert prevTotals[j] == inputs[0];
        }
      }
    }
  }
}
```

```

    } else {
      assert 0 <= j - 1 < k;
      assert totals[j - 1] == prevTotals[j - 1];
      assert prevTotals[j] == prevTotals[j - 1] + inputs[j];
    }
  } else {
    assert j == k;
    assert 0 < j;
    assert j - 1 == k - 1;
    assert totals[j] == next;
    assert totals[j - 1] == prevTotals[k - 1];
    assert next == prevTotals[k - 1] + inputs[k];
  }
}

k := k + 1;
}
}

```

Final Program

```

method p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)
  requires 0 < |inputs|
  ensures |totals| == |inputs|
  ensures forall i :: 0 <= i < |inputs| ==> totals[i] == (if i == 0 then inputs[0]
    ↪ else totals[i - 1] + inputs[i])
{
  totals := [inputs[0]];
  var n := |inputs|;
  var k := 1;

  while k < n
    invariant n == |inputs|
    invariant 1 <= k <= n
    invariant |totals| == k
    invariant forall j :: 0 <= j < k ==> totals[j] == (if j == 0 then inputs[0] else
      ↪ totals[j - 1] + inputs[j])
    decreases n - k
  {
    var prevTotals := totals;
    assert |prevTotals| == k;
    assert forall j :: 0 <= j < k ==> prevTotals[j] == (if j == 0 then inputs[0] else
      ↪ prevTotals[j - 1] + inputs[j]);

    var next := prevTotals[k - 1] + inputs[k];
    totals := prevTotals + [next];

    forall j | 0 <= j < k
      ensures totals[j] == prevTotals[j]
    {

```

```

    assert 0 <= j < |prevTotals|;
    assert totals == prevTotals + [next];
    assert totals[j] == (prevTotals + [next])[j];
    assert (prevTotals + [next])[j] == prevTotals[j];
}

forall j | 0 <= j < k + 1
  ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
{
  if j < k {
    assert 0 <= j < k;
    assert totals[j] == prevTotals[j];
    assert prevTotals[j] == (if j == 0 then inputs[0] else prevTotals[j - 1] + inputs[
      ↪ j]);
    if j == 0 {
      assert prevTotals[j] == inputs[0];
    } else {
      assert 0 <= j - 1 < k;
      assert totals[j - 1] == prevTotals[j - 1];
      assert prevTotals[j] == prevTotals[j - 1] + inputs[j];
    }
  } else {
    assert j == k;
    assert 0 < j;
    assert j - 1 == k - 1;
    assert totals[j] == next;
    assert totals[j - 1] == prevTotals[k - 1];
    assert next == prevTotals[k - 1] + inputs[k];
  }
}

k := k + 1;
}
}

```

Total Token Usage

Input tokens: 1374

Output tokens: 5997

Reasoning tokens: 4807

Sum of ‘total tokens’: 7371

Experiment Timings

Overall Experiment started at 1780652454921, ended at 1780652599541, lasting 144620ms (144.62 seconds)

Iteration #1 started at 1780652454921, ended at 1780652528040, lasting 73119ms (73.12 seconds)

Iteration #2 started at 1780652528040, ended at 1780652599540, lasting 71500ms (71.50 seconds)